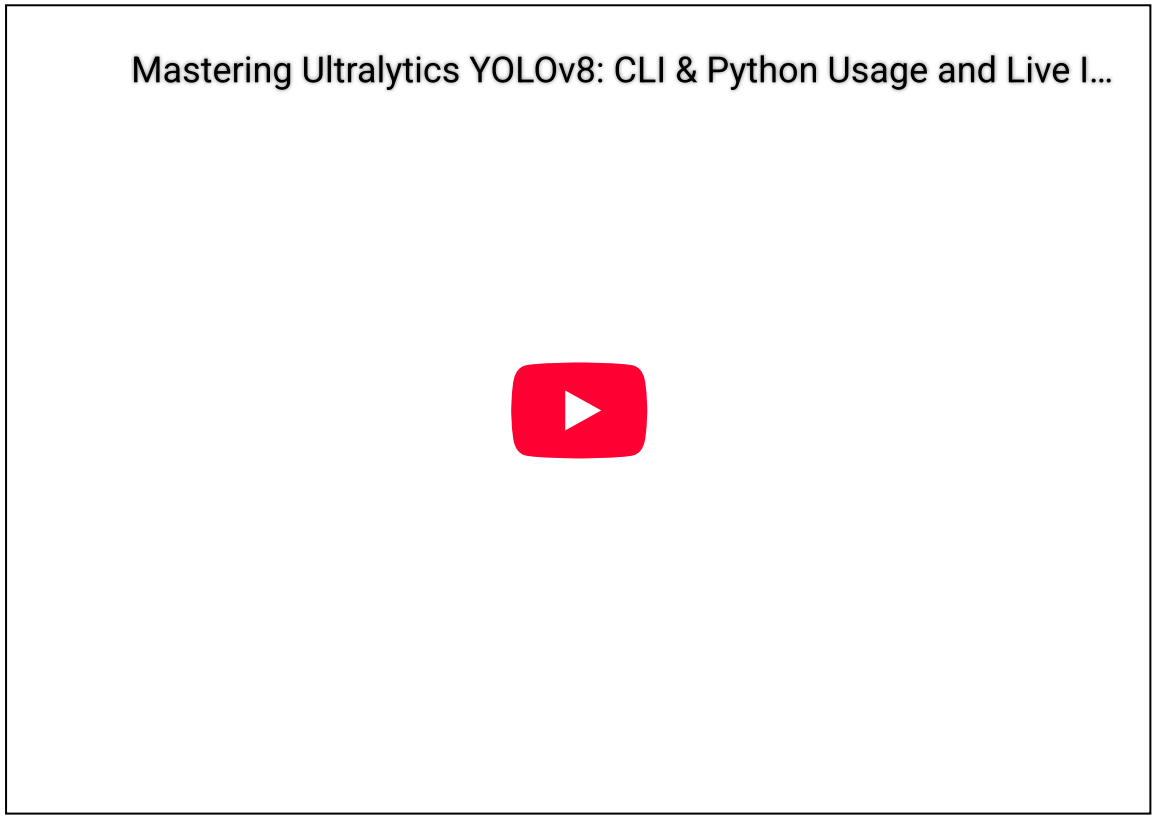
Configuration

YOLO settings and hyperparameters play a critical role in the model's performance, speed, and accuracy. These settings can affect the model's behavior at various stages, including training, validation, and prediction.

Watch: Mastering Ultralytics YOLO: Configuration



Watch: Mastering Ultralytics YOLO: Configuration

Ultralytics commands use the following syntax:

Example

CLI

```
yolo TASK MODE ARGS
```

Python

```
from ultralytics import YOLO

# Load a YOLO model from a pretrained weights file
model = YOLO("yolo26n.pt")

# Run the model in MODE using custom ARGS
MODE = "predict"
ARGS = {"source": "image.jpg", "imgsz": 640}
getattr(model, MODE)(**ARGS)
```

Where:

- `TASK` (optional) is one of (detect, segment, classify, pose, obb)
- `MODE` (required) is one of (train, val, predict, export, track, benchmark)
- `ARGS` (optional) are `arg=value` pairs like `imgsz=640` that override defaults.

Default `ARG` values are defined on this page and come from the `cfg/default.yaml` file.

Tasks

Ultralytics YOLO models can perform a variety of computer vision tasks, including:

- **Detect**: Object detection identifies and localizes objects within an image or video.
- **Segment**: Instance segmentation divides an image or video into regions corresponding to different objects or classes.
- **Classify**: Image classification predicts the class label of an input image.
- **Pose**: Pose estimation identifies objects and estimates their keypoints in an image or video.
- **OBB**: Oriented Bounding Boxes uses rotated bounding boxes, suitable for satellite or medical imagery.

Argument	Default	Description
`task`	`'detect'`	Specifies the YOLO task: `detect` for object detection, `segment` for segmentation, `classify` for classification, `pose` for pose estimation, and `obb` for oriented bounding boxes. Each task is tailored to specific outputs and problems in image and video analysis.

Tasks Guide

Modes

Ultralytics YOLO models operate in different modes, each designed for a specific stage of the model lifecycle:

- **Train**: Train a YOLO model on a custom dataset.
- **Val**: Validate a trained YOLO model.
- **Predict**: Use a trained YOLO model to make predictions on new images or videos.
- **Export**: Export a YOLO model for deployment.
- **Track**: Track objects in real-time using a YOLO model.
- **Benchmark**: Benchmark the speed and accuracy of YOLO exports (ONNX, TensorRT, etc.).

Argument	Default	Description
`mode`	`'train'`	Specifies the YOLO model's operating mode: `train` for model training, `val` for validation, `predict` for inference, `export` for converting to deployment formats, `track` for object tracking, and `benchmark` for performance evaluation. Each mode supports different stages, from development to deployment.

Modes Guide

Train Settings

Training settings for YOLO models include hyperparameters and configurations that affect the model's performance, speed, and accuracy. Key settings include batch size, learning rate, momentum, and weight decay. The choice of optimizer, loss function, and dataset composition also impact training. Tuning and experimentation are crucial for optimal performance. For more details, see the Ultralytics entrypoint function.

Argument	Type	Default	Description
`amp`	`bool`	`True`	Enables Automatic Mixed Precision (AMP) training, reducing memory usage and possibly speeding up training with minimal impact on accuracy.
`angle`	`float`	`1.0`	Weight of the angle loss in obb models, affecting the precision of oriented bounding box angle predictions.
`batch`	`int` or `float`	`16`	Batch size, with three modes: set as an integer (e.g., `batch=16`), auto mode for 60% GPU memory utilization (`batch=-1`), or auto mode with specified utilization fraction (`batch=0.70`).
`box`	`float`	`7.5`	Weight of the box loss component in the loss function, influencing how much emphasis is placed on accurately predicting bounding box coordinates.
`cache`	`bool`	`False`	Enables caching of dataset images in memory (`True`/`ram`), on disk (`disk`), or disables it (`False`). Improves training speed by reducing disk I/O at the cost of increased memory usage.
`classes`	`list[int]`	`None`	Specifies a list of class IDs to train on. Useful for filtering out and focusing only on certain classes during training.
`close_mosaic`	`int`	`10`	Disables mosaic data augmentation in the last N epochs to stabilize training before completion. Setting to 0 disables this feature.
`cls`	`float`	`0.5`	Weight of the classification loss in the total loss function, affecting the importance of correct class prediction relative to other components.

Argument	Type	Default	Description
`compile`	`bool` or `str`	`False`	Enables PyTorch 2.x `torch.compile` graph compilation with `backend='inductor'`. Accepts `True` → `"default"`, `False` → disables, or a string mode such as `"default"`, `"reduce-overhead"`, `"max-autotune-no-cudagraphs"`. Falls back to eager with a warning if unsupported.
`cos_lr`	`bool`	`False`	Utilizes a cosine learning rate scheduler, adjusting the learning rate following a cosine curve over epochs. Helps in managing learning rate for better convergence.
`data`	`str`	`None`	Path to the dataset configuration file (e.g., `coco8.yaml`). This file contains dataset-specific parameters, including paths to training and validation data, class names, and number of classes.
`deterministic`	`bool`	`True`	Forces deterministic algorithm use, ensuring reproducibility but may affect performance and speed due to the restriction on non-deterministic algorithms.
`device`	`int` or `str` or `list`	`None`	Specifies the computational device(s) for training: a single GPU (`device=0`), multiple GPUs (`device=[0,1]`), CPU (`device=cpu`), MPS for Apple silicon (`device=mps`), or auto-selection of most idle GPU (`device=-1`) or multiple idle GPUs (`device=[-1,-1]`)
`dfl`	`float`	`1.5`	Weight of the distribution focal loss, used in certain YOLO versions for fine-grained classification.
`dropout`	`float`	`0.0`	Dropout rate for regularization in classification tasks, preventing overfitting by randomly omitting units during training.
`epochs`	`int`	`100`	Total number of training epochs. Each epoch represents a full pass over the entire dataset. Adjusting this value can affect training duration and model performance.

Argument	Type	Default	Description
`exist_ok`	`bool`	`False`	If True, allows overwriting of an existing project/name directory. Useful for iterative experimentation without needing to manually clear previous outputs.
`fraction`	`float`	`1.0`	Specifies the fraction of the dataset to use for training. Allows for training on a subset of the full dataset, useful for experiments or when resources are limited.
`freeze`	`int` or `list`	`None`	Freezes the first N layers of the model or specified layers by index, reducing the number of trainable parameters. Useful for fine-tuning or transfer learning.
`imgsz`	`int`	`640`	Target image size for training. Images are resized to squares with sides equal to the specified value (if `rect=False`), preserving aspect ratio for YOLO models but not RT-DETR. Affects model accuracy and computational complexity.
`kobj`	`float`	`1.0`	Weight of the keypoint objectness loss in pose estimation models, balancing detection confidence with pose accuracy.
`lr0`	`float`	`0.01`	Initial learning rate (i.e. `SGD=1E-2`, `Adam=1E-3`). Adjusting this value is crucial for the optimization process, influencing how rapidly model weights are updated.
`lrf`	`float`	`0.01`	Final learning rate as a fraction of the initial rate = (`lr0 * lrf`), used in conjunction with schedulers to adjust the learning rate over time.
`mask_ratio`	`int`	`4`	Downsample ratio for segmentation masks, affecting the resolution of masks used during training.
`max_det`	`int`	`300`	Specifies the maximum number of objects retained during validation phase of training.

Argument	Type	Default	Description
`model`	`str`	`None`	Specifies the model file for training. Accepts a path to either a `.pt` pretrained model or a `.yaml` configuration file. Essential for defining the model structure or initializing weights.
`momentum`	`float`	`0.937`	Momentum factor for SGD or beta1 for Adam optimizers, influencing the incorporation of past gradients in the current update.
`multi_scale`	`float`	`0.0`	Randomly vary `imgsz` each batch by +/- `multi_scale` (e.g. `0.25` -> `0.75x` to `1.25x`), rounding to model stride multiples; `0.0` disables multi-scale training.
`name`	`str`	`None`	Name of the training run. Used for creating a subdirectory within the project folder, where training logs and outputs are stored.
`nbs`	`int`	`64`	Nominal batch size for normalization of loss.
`optimizer`	`str`	`'auto'`	Choice of optimizer for training. Options include `SGD`, `MuSGD`, `Adam`, `Adamax`, `AdamW`, `NAdam`, `RAdam`, `RMSProp`, or `auto` for automatic selection based on model configuration. Affects convergence speed and stability.
`overlap_mask`	`bool`	`True`	Determines whether object masks should be merged into a single mask for training, or kept separate for each object. In case of overlap, the smaller mask is overlaid on top of the larger mask during merge.
`patience`	`int`	`100`	Number of epochs to wait without improvement in validation metrics before early stopping the training. Helps prevent overfitting by stopping training when performance plateaus.
`plots`	`bool`	`True`	Generates and saves plots of training and validation metrics, as well as prediction examples, providing visual insights into model performance and learning progression.

Argument	Type	Default	Description
`pose`	`float`	`12.0`	Weight of the pose loss in models trained for pose estimation, influencing the emphasis on accurately predicting pose keypoints.
`pretrained`	`bool` or `str`	`True`	Determines whether to start training from a pretrained model. Can be a boolean value or a string path to a specific model from which to load weights. Enhances training efficiency and model performance.
`profile`	`bool`	`False`	Enables profiling of ONNX and TensorRT speeds during training, useful for optimizing model deployment.
`project`	`str`	`None`	Name of the project directory where training outputs are saved. Allows for organized storage of different experiments.
`rect`	`bool`	`False`	Enables minimum padding strategy—images in a batch are minimally padded to reach a common size, with the longest side equal to `imgsz`. Can improve efficiency and speed but may affect model accuracy.
`resume`	`bool`	`False`	Resumes training from the last saved checkpoint. Automatically loads model weights, optimizer state, and epoch count, continuing training seamlessly.
`rle`	`float`	`1.0`	Weight of the residual log-likelihood estimation loss in pose estimation models, affecting the precision of keypoint localization.
`save`	`bool`	`True`	Enables saving of training checkpoints and final model weights. Useful for resuming training or model deployment.
`save_period`	`int`	`-1`	Frequency of saving model checkpoints, specified in epochs. A value of -1 disables this feature. Useful for saving interim models during long training sessions.
`seed`	`int`	`0`	Sets the random seed for training, ensuring reproducibility of results

Argument	Type	Default	Description
			across runs with the same configurations.
`single_cls`	`bool`	`False`	Treats all classes in multi-class datasets as a single class during training. Useful for binary classification tasks or when focusing on object presence rather than classification.
`time`	`float`	`None`	Maximum training time in hours. If set, this overrides the `epochs` argument, allowing training to automatically stop after the specified duration. Useful for time-constrained training scenarios.
`val`	`bool`	`True`	Enables validation during training, allowing for periodic evaluation of model performance on a separate dataset.
`verbose`	`bool`	`True`	Enables verbose output during training, displaying progress bars, per-epoch metrics, and additional training information in the console.
`warmup_bias_lr`	`float`	`0.1`	Learning rate for bias parameters during the warmup phase, helping stabilize model training in the initial epochs.
`warmup_epochs`	`float`	`3.0`	Number of epochs for learning rate warmup, gradually increasing the learning rate from a low value to the initial learning rate to stabilize training early on.
`warmup_momentum`	`float`	`0.8`	Initial momentum for warmup phase, gradually adjusting to the set momentum over the warmup period.
`weight_decay`	`float`	`0.0005`	L2 regularization term, penalizing large weights to prevent overfitting.
`workers`	`int`	`8`	Number of worker threads for data loading (per `RANK` if Multi-GPU training). Influences the speed of data preprocessing and feeding into the model, especially useful in multi-GPU setups.

Note on Batch-size Settings

The `batch` argument offers three configuration options:

- **Fixed Batch Size**: Specify the number of images per batch with an integer (e.g., `batch=16`).
- **Auto Mode (60% GPU Memory)**: Use `batch=-1` for automatic adjustment to approximately 60% CUDA memory utilization.
- **Auto Mode with Utilization Fraction**: Set a fraction (e.g., `batch=0.70`) to adjust based on a specified GPU memory usage.

Train Guide

Predict Settings

Prediction settings for YOLO models include hyperparameters and configurations that influence performance, speed, and accuracy during inference. Key settings include the confidence threshold, Non-Maximum Suppression (NMS) threshold, and the number of classes. Input data size, format, and supplementary features like masks also affect predictions. Tuning these settings is essential for optimal performance.

Inference arguments:

Argument	Type	Default	Description
`source`	`str`	`'ultralytics/assets'`	Specifies the data source for inference. Can be an image path, video file, directory, URL, or device ID for live feeds. Supports a wide range of formats and sources, enabling flexible application across different types of input.
`conf`	`float`	`0.25`	Sets the minimum confidence threshold for detections. Objects detected with confidence below this threshold will be disregarded. Adjusting this value can help reduce false positives.
`iou`	`float`	`0.7`	Intersection Over Union (IoU) threshold for Non-Maximum Suppression (NMS). Lower values result in fewer detections by

Argument	Type	Default	Description
			eliminating overlapping boxes, useful for reducing duplicates.
`imgsz`	`int` or `tuple`	`640`	Defines the image size for inference. Can be a single integer `640` for square resizing or a (height, width) tuple. Proper sizing can improve detection accuracy and processing speed.
`rect`	`bool`	`True`	If enabled, minimally pads the shorter side of the image until it's divisible by stride to improve inference speed. If disabled, pads the image to a square during inference.
`half`	`bool`	`False`	Enables half-precision (FP16) inference, which can speed up model inference on supported GPUs with minimal impact on accuracy.
`device`	`str`	`None`	Specifies the device for inference (e.g., `cpu`, `cuda:0` or `0`). Allows users to select between CPU, a specific GPU, or other compute devices for model execution.
`batch`	`int`	`1`	Specifies the batch size for inference (only works when the source is a directory, video file, or `.txt` file). A larger batch size can provide higher throughput, shortening the total amount of time required for inference.
`max_det`	`int`	`300`	Maximum number of detections allowed per image. Limits the total number of objects the model can detect in a single inference, preventing excessive outputs in dense scenes.
`vid_stride`	`int`	`1`	Frame stride for video inputs. Allows skipping frames in videos to speed up processing at the cost of temporal resolution. A value of 1 processes every frame, higher values skip frames.
`stream_buffer`	`bool`	`False`	Determines whether to queue incoming frames for video streams. If `False`, old frames get dropped to accommodate new frames (optimized for real-time applications). If `True`, queues new

Argument	Type	Default	Description
			frames in a buffer, ensuring no frames get skipped, but will cause latency if inference FPS is lower than stream FPS.
`visualize`	`bool`	`False`	Activates visualization of model features during inference, providing insights into what the model is "seeing". Useful for debugging and model interpretation.
`augment`	`bool`	`False`	Enables test-time augmentation (TTA) for predictions, potentially improving detection robustness at the cost of inference speed.
`agnostic_nms`	`bool`	`False`	Enables class-agnostic Non-Maximum Suppression (NMS), which merges overlapping boxes of different classes. Useful in multi-class detection scenarios where class overlap is common. For end-to-end models (YOLO26, YOLOv10), this only prevents the same detection from appearing with multiple class labels (IoU=1.0 duplicates) and does not perform IoU-threshold-based suppression between distinct boxes.
`classes`	`list[int]`	`None`	Filters predictions to a set of class IDs. Only detections belonging to the specified classes will be returned. Useful for focusing on relevant objects in multi-class detection tasks.
`retina_masks`	`bool`	`False`	Returns high-resolution segmentation masks. The returned masks (`masks.data`) will match the original image size if enabled. If disabled, they have the image size used during inference.
`embed`	`list[int]`	`None`	Specifies the layers from which to extract feature vectors or embeddings. Useful for downstream tasks like clustering or similarity search.
`project`	`str`	`None`	Name of the project directory where prediction outputs are saved if `save` is enabled.
`name`	`str`	`None`	Name of the prediction run. Used for creating a subdirectory within the project folder, where prediction

Argument	Type	Default	Description
			outputs are stored if `save` is enabled.
`stream`	`bool`	`False`	Enables memory-efficient processing for long videos or numerous images by returning a generator of Results objects instead of loading all frames into memory at once.
`verbose`	`bool`	`True`	Controls whether to display detailed inference logs in the terminal, providing real-time feedback on the prediction process.
`compile`	`bool` or `str`	`False`	Enables PyTorch 2.x `torch.compile` graph compilation with `backend='inductor'`. Accepts `True` → `"default"`, `False` → disables, or a string mode such as `"default"`, `"reduce-overhead"`, `"max-autotune-no-cudagraphs"`. Falls back to eager with a warning if unsupported.
`end2end`	`bool`	`None`	Overrides the end-to-end mode in YOLO models that support NMS-free inference (YOLO26, YOLOv10). Setting it to `False` lets you run prediction using the traditional NMS pipeline, additionally allowing you to make use of the `iou` argument.

Visualization arguments:

Argument	Type	Default	Description
`show`	`bool`	`False`	If `True`, displays the annotated images or videos in a window. Useful for immediate visual feedback during development or testing.
`save`	`bool`	`False or True`	Enables saving of the annotated images or videos to files. Useful for documentation, further analysis, or sharing results. Defaults to True when using CLI & False when used in Python.

Argument	Type	Default	Description
`save_frames`	`bool`	`False`	When processing videos, saves individual frames as images. Useful for extracting specific frames or for detailed frame-by-frame analysis.
`save_txt`	`bool`	`False`	Saves detection results in a text file, following the format `[class] [x_center] [y_center] [width] [height] [confidence]`. Useful for integration with other analysis tools.
`save_conf`	`bool`	`False`	Includes confidence scores in the saved text files. Enhances the detail available for post-processing and analysis.
`save_crop`	`bool`	`False`	Saves cropped images of detections. Useful for dataset augmentation, analysis, or creating focused datasets for specific objects.
`show_labels`	`bool`	`True`	Displays labels for each detection in the visual output. Provides immediate understanding of detected objects.
`show_conf`	`bool`	`True`	Displays the confidence score for each detection alongside the label. Gives insight into the model's certainty for each detection.
`show_boxes`	`bool`	`True`	Draws bounding boxes around detected objects. Essential for visual identification and location of objects in images or video frames.
`line_width`	`int or None`	`None`	Specifies the line width of bounding boxes. If `None`, the line width is automatically adjusted based on the image size. Provides visual customization for clarity.

Predict Guide

Validation Settings

Validation settings for YOLO models involve hyperparameters and configurations to evaluate performance on a validation dataset. These settings influence performance, speed, and accuracy. Common settings include batch size, validation frequency, and

performance metrics. The validation dataset's size and composition, along with the specific task, also affect the process.

Argument	Type	Default	Description
`data`	`str`	`None`	Specifies the path to the dataset configuration file (e.g., `coco8.yaml`). This file should include the path to the validation data.
`imgsz`	`int`	`640`	Defines the size of input images. All images are resized to this dimension before processing. Larger sizes may improve accuracy for small objects but increase computation time.
`batch`	`int`	`16`	Sets the number of images per batch. Higher values utilize GPU memory more efficiently but require more VRAM. Adjust based on available hardware resources.
`save_json`	`bool`	`False`	If `True`, saves the results to a JSON file for further analysis, integration with other tools, or submission to evaluation servers like COCO.
`conf`	`float`	`0.001`	Sets the minimum confidence threshold for detections. Lower values increase recall but may introduce more false positives. Used during validation to compute precision-recall curves.
`iou`	`float`	`0.7`	Sets the Intersection Over Union threshold for Non-Maximum Suppression. Controls duplicate detection elimination.
`max_det`	`int`	`300`	Limits the maximum number of detections per image. Useful in dense scenes to prevent excessive detections and manage computational resources.
`half`	`bool`	`False`	Enables half-precision (FP16) computation, reducing memory usage and potentially increasing speed with minimal impact on accuracy.

Argument	Type	Default	Description
`device`	`str`	`None`	Specifies the device for validation (`cpu`, `cuda:0`, etc.). When `None`, automatically selects the best available device. Multiple CUDA devices can be specified with comma separation.
`dnn`	`bool`	`False`	If `True`, uses the OpenCV DNN module for ONNX model inference, offering an alternative to PyTorch inference methods.
`plots`	`bool`	`True`	When set to `True`, generates and saves plots of predictions versus ground truth, confusion matrices, and PR curves for visual evaluation of model performance.
`classes`	`list[int]`	`None`	Specifies a list of class IDs to evaluate. Useful for filtering out and focusing only on certain classes during evaluation.
`rect`	`bool`	`True`	If `True`, uses rectangular inference for batching, reducing padding and potentially increasing speed and efficiency by processing images in their original aspect ratio.
`split`	`str`	`'val'`	Determines the dataset split to use for validation (`val`, `test`, or `train`). Allows flexibility in choosing the data segment for performance evaluation.
`project`	`str`	`None`	Name of the project directory where validation outputs are saved. Helps organize results from different experiments or models.
`name`	`str`	`None`	Name of the validation run. Used for creating a subdirectory within the project folder, where validation logs and outputs are stored.
`verbose`	`bool`	`True`	If `True`, displays detailed information during the validation process, including per-class metrics, batch progress, and additional debugging information.

Argument	Type	Default	Description
`save_txt`	`bool`	`False`	If `True`, saves detection results in text files, with one file per image, useful for further analysis, custom post-processing, or integration with other systems.
`save_conf`	`bool`	`False`	If `True`, includes confidence values in the saved text files when `save_txt` is enabled, providing more detailed output for analysis and filtering.
`workers`	`int`	`8`	Number of worker threads for data loading. Higher values can speed up data preprocessing but may increase CPU usage. Setting to 0 uses main thread, which can be more stable in some environments.
`augment`	`bool`	`False`	Enables test-time augmentation (TTA) during validation, potentially improving detection accuracy at the cost of inference speed by running inference on transformed versions of the input.
`agnostic_nms`	`bool`	`False`	Enables class-agnostic Non-Maximum Suppression, which merges overlapping boxes regardless of their predicted class. Useful for instance-focused applications. For end-to-end models (YOLO26, YOLOv10), this only prevents the same detection from appearing with multiple class labels (IoU=1.0 duplicates) and does not perform IoU-threshold-based suppression between distinct boxes.
`single_cls`	`bool`	`False`	Treats all classes as a single class during validation. Useful for evaluating model performance on binary detection tasks or when class distinctions aren't important.
`visualize`	`bool`	`False`	Visualizes the ground truths, true positives, false positives, and false negatives for each image. Useful for debugging and model interpretation.
`compile`	`bool` or `str`	`False`	Enables PyTorch 2.x `torch.compile` graph compilation with `backend='inductor'`. Accepts `True` → `"default"`, `False` → disables, or a string mode such as `"default"`, `"reduce-overhead"`, `"max-autotune-`

Argument	Type	Default	Description
			`no-cudagraphs"`. Falls back to eager with a warning if unsupported.
`end2end`	`bool`	`None`	Overrides the end-to-end mode in YOLO models that support NMS-free inference (YOLO26, YOLOv10). Setting it to `False` lets you run validation using the traditional NMS pipeline, additionally allowing you to make use of the `iou` argument.

Careful tuning and experimentation are crucial to ensure optimal performance and to detect and prevent overfitting.

Val Guide

Export Settings

Export settings for YOLO models include configurations for saving or exporting the model for use in different environments. These settings impact performance, size, and compatibility. Key settings include the exported file format (e.g., ONNX, TensorFlow SavedModel), the target device (e.g., CPU, GPU), and features like masks. The model's task and the destination environment's constraints also affect the export process.

Argument	Type	Default	Description
`format`	`str`	`'torchscript'`	Target format for the exported model, such as `'onnx'`, `'torchscript'`, `'engine'` (TensorRT), or others. Each format enables compatibility with different deployment environments.
`imgsz`	`int` or `tuple`	`640`	Desired image size for the model input. Can be an integer for square images (e.g., `640` for 640×640) or a tuple `(height, width)` for specific dimensions.
`keras`	`bool`	`False`	Enables export to Keras format for TensorFlow SavedModel, providing compatibility with TensorFlow serving and APIs.
`optimize`	`bool`	`False`	Applies optimization for mobile devices when exporting to TorchScript, potentially reducing

Argument	Type	Default	Description
			model size and improving inference performance. Not compatible with NCNN format or CUDA devices.
`half`	`bool`	`False`	Enables FP16 (half-precision) quantization, reducing model size and potentially speeding up inference on supported hardware. Not compatible with INT8 quantization or CPU-only exports. Only available for certain formats, e.g. ONNX (see below).
`int8`	`bool`	`False`	Activates INT8 quantization, further compressing the model and speeding up inference with minimal accuracy loss, primarily for edge devices. When used with TensorRT, performs post-training quantization (PTQ).
`dynamic`	`bool`	`False`	Allows dynamic input sizes for TorchScript, ONNX, OpenVINO, TensorRT, and CoreML exports, enhancing flexibility in handling varying image dimensions. Automatically set to `True` when using TensorRT with INT8.
`simplify`	`bool`	`True`	Simplifies the model graph for ONNX exports with `onnxslim`, potentially improving performance and compatibility with inference engines.
`opset`	`int`	`None`	Specifies the ONNX opset version for compatibility with different ONNX parsers and runtimes. If not set, uses the latest supported version.
`workspace`	`float` or `None`	`None`	Sets the maximum workspace size in GiB for TensorRT optimizations, balancing memory usage and performance. Use `None` for auto-allocation by TensorRT up to device maximum.
`nms`	`bool`	`False`	Adds Non-Maximum Suppression (NMS) to the exported model when supported (see Export Formats), improving detection post-processing efficiency. Not available for end2end models.
`batch`	`int`	`1`	Specifies export model batch inference size or the maximum

Argument	Type	Default	Description
			number of images the exported model will process concurrently in `predict` mode. For Edge TPU exports, this is automatically set to 1.
`device`	`str`	`None`	Specifies the device for exporting: GPU (`device=0`), CPU (`device=cpu`), MPS for Apple silicon (`device=mps`) or DLA for NVIDIA Jetson (`device=dla:0` or `device=dla:1`). TensorRT exports automatically use GPU.
`data`	`str`	`'coco8.yaml'`	Path to the dataset configuration file, essential for INT8 quantization calibration. If not specified with INT8 enabled, `coco8.yaml` will be used as a fallback for calibration.
`fraction`	`float`	`1.0`	Specifies the fraction of the dataset to use for INT8 quantization calibration. Allows for calibrating on a subset of the full dataset, useful for experiments or when resources are limited. If not specified with INT8 enabled, the full dataset will be used.
`end2end`	`bool`	`None`	Overrides the end-to-end mode in YOLO models that support NMS-free inference (YOLO26, YOLOv10). Setting it to `False` lets you export these models to be compatible with the traditional NMS-based postprocessing pipeline.

Thoughtful configuration ensures the exported model is optimized for its use case and functions effectively in the target environment.

Export Guide

Solutions Settings

Ultralytics Solutions configuration settings offer flexibility to customize models for tasks like object counting, heatmap creation, workout tracking, data analysis, zone tracking, queue management, and region-based counting. These options allow easy adjustments for accurate and useful results tailored to specific needs.

Argument	Type	Default	Description
`model`	`str`	`None`	Path to an Ultralytics YOLO model file.
`region`	`list`	`'[(20, 400), (1260, 400)]'`	List of points defining the counting region.
`show_in`	`bool`	`True`	Flag to control whether to display the in counts on the video stream.
`show_out`	`bool`	`True`	Flag to control whether to display the out counts on the video stream.
`analytics_type`	`str`	`'line'`	Type of graph, i.e., `line`, `bar`, `area`, or `pie`.
`colormap`	`int`	`cv2.COLORMAP_JET`	Colormap to use for the heatmap.
`json_file`	`str`	`None`	Path to the JSON file that contains all parking coordinates data.
`up_angle`	`float`	`145.0`	Angle threshold for the 'up' pose.
`kpts`	`list[int]`	`'[6, 8, 10]'`	List of three keypoint indices used for monitoring workouts. These keypoints correspond to body joints or parts, such as shoulders, elbows, and wrists, for exercises like push-ups, pull-ups, squats, and ab-workouts.
`down_angle`	`float`	`90.0`	Angle threshold for the 'down' pose.
`blur_ratio`	`float`	`0.5`	Adjusts percentage of blur intensity, with values in range `0.1 - 1.0`.
`crop_dir`	`str`	`'cropped-detections'`	Directory name for storing cropped detections.
`records`	`int`	`5`	Total detections count to trigger an email with security alarm system.
`vision_point`	`tuple[int, int]`	`(20, 20)`	The point where vision will track objects and draw paths using VisionEye Solution.

Argument	Type	Default	Description
`source`	`str`	`None`	Path to the input source (video, RTSP, etc.). Only usable with Solutions command line interface (CLI).
`figsize`	`tuple[int, int]`	`(12.8, 7.2)`	Figure size for analytics charts such as heatmaps or graphs.
`fps`	`float`	`30.0`	Frames per second used for speed calculations.
`max_hist`	`int`	`5`	Maximum historical points to track per object for speed/direction calculations.
`meter_per_pixel`	`float`	`0.05`	Scaling factor used for converting pixel distance to real-world units.
`max_speed`	`int`	`120`	Maximum speed limit in visual overlays (used in alerts).
`data`	`str`	`'images'`	Path to image directory used for similarity search.

Solutions Guide

Augmentation Settings

Data augmentation techniques are essential for improving YOLO model robustness and performance by introducing variability into the training data, helping the model generalize better to unseen data. The following table outlines each augmentation argument's purpose and effect:

Argument	Type	Default	Supported Tasks	Range	Description
`hsv_h`	`float`	`0.015`	`detect`, `segment`, `pose`, `obb`, `classify`	`0.0 - 1.0`	Adjusts the of the imag a fraction o the color wheel, introducing color variability. Helps the model generalize across

Argument	Type	Default	Supported Tasks	Range	Description
					different lighting conditions.
`hsv_s`	`float`	`0.7`	`detect`, `segment`, `pose`, `obb`, `classify`	`0.0 - 1.0`	Alters the saturation c the image b fraction, affecting th intensity of colors. Use for simulati different environmen conditions.
`hsv_v`	`float`	`0.4`	`detect`, `segment`, `pose`, `obb`, `classify`	`0.0 - 1.0`	Modifies th value (brightness the image b fraction, helping the model to perform we under vario lighting conditions.
`degrees`	`float`	`0.0`	`detect`, `segment`, `pose`, `obb`	`0.0 - 180`	Rotates the image randomly within the specified degree ran improving t model's abi to recogniz objects at various orientations
`translate`	`float`	`0.1`	`detect`, `segment`, `pose`, `obb`	`0.0 - 1.0`	Translates t image horizontally and vertical by a fractio the image s aiding in learning to detect parti visible obje
`scale`	`float`	`0.5`	`detect`, `segment`, `pose`, `obb`, `classify`	`0 - 1`	Scales the image by a gain factor, simulating objects at different

Argument	Type	Default	Supported Tasks	Range	Description
					distances fr the camera
`shear`	`float`	`0.0`	`detect`, `segment`, `pose`, `obb`	`-180 - +180`	Shears the image by a specified degree, mimicking t effect of objects bei viewed fron different angles.
`perspectiv e`	`float`	`0.0`	`detect`, `segment`, `pose`, `obb`	`0.0 - 0.001`	Applies a random perspective transformat to the imag enhancing t model's abi to understa objects in 3 space.
`flipud`	`float`	`0.0`	`detect`, `segment`, `pose`, `obb`, `classify`	`0.0 - 1.0`	Flips the im upside dow with the specified probability, increasing t data variabi without affecting th object's characteris
`fliplr`	`float`	`0.5`	`detect`, `segment`, `pose`, `obb`, `classify`	`0.0 - 1.0`	Flips the im left to right the specifie probability, useful for learning symmetrica objects and increasing dataset diversity.
`bgr`	`float`	`0.0`	`detect`, `segment`, `pose`, `obb`	`0.0 - 1.0`	Flips the im channels fr RGB to BGR with the specified probability, useful for increasing robustness

Argument	Type	Default	Supported Tasks	Range	Description
					incorrect channel ordering.
`mosaic`	`float`	`1.0`	`detect`, `segment`, `pose`, `obb`	`0.0 - 1.0`	Combines f training ima into one, simulating different sc compositio and object interactions Highly effe for complex scene understand
`mixup`	`float`	`0.0`	`detect`, `segment`, `pose`, `obb`	`0.0 - 1.0`	Blends two images and their labels, creating a composite image. Enhances t model's abi to generaliz by introduc label noise visual variability.
`cutmix`	`float`	`0.0`	`detect`, `segment`, `pose`, `obb`	`0.0 - 1.0`	Combines portions of images, creating a partial blen while maintaining distinct regions. Enhances model robustness creating occlusion scenarios.
`copy_past e`	`float`	`0.0`	`segment`	`0.0 - 1.0`	Copies and pastes obje across imag to increase object instances.
`copy_paste _mode`	`str`	`flip`	`segment`	-	Specifies th `copy-paste` strategy to Options

Argument	Type	Default	Supported Tasks	Range	Description
					include 'fl and 'mixup
auto_augment	str	randaugment	classify	-	Applies a predefined augmentati policy ('randaugme ', 'autoaugme , or 'augmi to enhance model performanc through visu diversity.
erasing	float	0.4	classify	0.0 - 1.0	Randomly erases regi of the imag during train to encourag the model t focus on les obvious features.
augmentations	list	``	detect, segment, pose, obb	-	Custom Albumentat transforms advanced d augmentati (Python API only). Acce a list of transform objects for specialized augmentati needs.

Adjust these settings to meet dataset and task requirements. Experimenting with different values can help find the optimal augmentation strategy for the best model performance.

Augmentation Guide

Logging, Checkpoints and Plotting Settings

Logging, checkpoints, plotting, and file management are important when training a YOLO model:

- **Logging**: Track the model's progress and diagnose issues using libraries like TensorBoard or by writing to a file.
- **Checkpoints**: Save the model at regular intervals to resume training or experiment with different configurations.
- **Plotting**: Visualize performance and training progress using libraries like matplotlib or TensorBoard.
- **File management**: Organize files generated during training, such as checkpoints, log files, and plots, for easy access and analysis.

Effective management of these aspects helps track progress and makes debugging and optimization easier.

Argument	Default	Description
`project`	`'runs'`	Specifies the root directory for saving training runs. Each run is saved in a separate subdirectory.
`name`	`'exp'`	Defines the experiment name. If unspecified, YOLO increments this name for each run (e.g., `exp`, `exp2`) to avoid overwriting.
`exist_ok`	`False`	Determines whether to overwrite an existing experiment directory. `True` allows overwriting; `False` prevents it.
`plots`	`False`	Controls the generation and saving of training and validation plots. Set to `True` to create plots like loss curves, precision-recall curves, and sample predictions for visual tracking of performance.
`save`	`False`	Enables saving training checkpoints and final model weights. Set to `True` to save model states periodically, allowing training resumption or model deployment.

FAQ

How do I improve my YOLO model's performance during training?

Improve performance by tuning hyperparameters like batch size, learning rate, momentum, and weight decay. Adjust data augmentation settings, select the right optimizer, and use techniques like early stopping or mixed precision. For details, see the Train Guide.

What are the key hyperparameters for YOLO model accuracy?

Key hyperparameters affecting accuracy include:

- **Batch Size (`batch`)**: Larger sizes can stabilize training but need more memory.
- **Learning Rate (`lr0`)**: Smaller rates offer fine adjustments but slower convergence.
- **Momentum (`momentum`)**: Accelerates gradient vectors, dampening oscillations.
- **Image Size (`imgsz`)**: Larger sizes improve accuracy but increase computational load.

Adjust these based on your dataset and hardware. Learn more in Train Settings.

How do I set the learning rate for training a YOLO model?

The learning rate (`lr0`) is crucial; start with `0.01` for SGD or `0.001` for Adam optimizer. Monitor metrics and adjust as needed. Use cosine learning rate schedulers (`cos_lr`) or warmup (`warmup_epochs`, `warmup_momentum`). Details are in the Train Guide.

What are the default inference settings for YOLO models?

Default settings include:

- **Confidence Threshold (`conf=0.25`)**: Minimum confidence for detections.
- **IoU Threshold (`iou=0.7`)**: For Non-Maximum Suppression (NMS).
- **Image Size (`imgsz=640`)**: Resizes input images.
- **Device (`device=None`)**: Selects CPU or GPU.

For a full overview, see Predict Settings and the Predict Guide.

Why use mixed precision training with YOLO models?

Mixed precision training (`amp=True`) reduces memory usage and speeds up training using FP16 and FP32. It's beneficial for modern GPUs, allowing larger models and faster computations without significant accuracy loss. Learn more in the Train Guide.

Tweet Share